

QueryFlix

Netflix Movies and TV Shows using SQL and Data Analysis

PBL-II — First Review: Design & Implementation Pack

Simran Sharma • Rishav Tandon • Siddharth Ganesh • Alishri Poddar

Supervisor: Dr. R. Sathya | Department of Computing Technologies

Status at a glance

Component	Status
Dataset	Real Kaggle/TMDB-enriched Netflix dataset (movies + TV shows, up to 2025) — 32,000 titles loaded
Database	PostgreSQL 16, single unified “netflix” table, indexed on type / release_year / date_added
SQL problems	15 / 15 written, executed, and verified against real data (see results below)
Techniques covered	Window functions, string unnesting, date operations, pattern matching, dynamic CASE categorization, aggregation
Remaining for final submission	Report write-up, results interpretation/insights narrative, final proofread of queries

Design & Data-Cleaning Decisions

The final dataset (real Kaggle/TMDB-sourced CSVs, 16,000 movies + 16,000 TV shows) differs in a few respects from the classic Netflix Kaggle set the proposal was originally scoped against. These are the adaptations made, and the reasoning — good material for viva questions.

Surrogate primary key

The source show_id repeats across the movies and TV shows files (397 overlapping IDs) and even within the TV shows file (9 duplicates), so it can't be a primary key on its own. We generate a SERIAL show_id and keep the original as source_id for traceability.

Duration field dropped from analysis

In this dataset, duration is empty for all 16,000 movies and a constant “1 Seasons” for all 16,000 TV shows — it carries no information. We noted this as a data-quality finding rather than silently ignoring it, and use vote_count / popularity as the meaningful engagement metrics instead.

Categorical maturity rating → numeric rating score

This dataset does not carry the classic PG-13 / TV-MA style rating; instead it has a TMDB-style numeric quality score (0–10), alongside popularity, vote_count, and vote_average. “Common ratings” analysis was reframed as rating-score and popularity analysis (Q7, Q8, Q10).

Movies-only financial fields

budget and revenue are only populated for movies. Q13 (profitability) is scoped to type = 'Movie' accordingly, with a NULLIF guard to avoid division by zero.

The 15 Business Problems — Implementation & Live Results

Every query below was executed against the real, loaded 32,000-row dataset just now; the tables show actual output (top rows only, for space).

Q1. Content-type distribution

Aggregation

Problem: How much of the catalog is Movies vs. TV Shows?

```
SELECT type, COUNT(*) AS total_titles,  
       ROUND(100.0*COUNT(*)/SUM(COUNT(*)) OVER (), 2) AS pct  
FROM netflix GROUP BY type;
```

type	total_titles	pct
Movie	16000	50.00
TV Show	16000	50.00

Q2. Top 10 genres by frequency

String manipulation –
STRING_TO_ARRAY / UNNEST

Problem: Which genres dominate the catalog? (genres field is multi-valued/comma-separated)

```
SELECT TRIM(genre), COUNT(*) FROM netflix,  
       UNNEST(STRING_TO_ARRAY(genres, ',')) AS genre  
GROUP BY TRIM(genre) ORDER BY 2 DESC LIMIT 10;
```

genre	title_count
Drama	14776
Comedy	9110
Animation	4070
Thriller	3769
Action	3239
Crime	3203

(10 rows returned; showing first 6)

Q3. Top 10 producing countries

String manipulation – UNNEST

Problem: Which countries produce the most Netflix content?

```
SELECT TRIM(c), COUNT(*) FROM netflix,  
       UNNEST(STRING_TO_ARRAY(country, ',')) AS c  
GROUP BY TRIM(c) ORDER BY 2 DESC LIMIT 10;
```

country	title_count
United States of America	10956
Japan	2871
United Kingdom	2609
China	2399
South Korea	2174
France	2108

(10 rows returned; showing first 6)

Q4. Top 10 most-credited cast members

String manipulation – UNNEST

Problem: Who are the most frequently credited actors across titles?

```
SELECT TRIM(actor), COUNT(*) FROM netflix,  
       UNNEST(STRING_TO_ARRAY(cast_members, ',')) AS actor  
GROUP BY TRIM(actor) ORDER BY 2 DESC LIMIT 10;
```

actor	appearances
Min Do-yoon	111
Kana Hanazawa	97
Yuki Kaji	70
Grey DeLisle	65
Takahiro Sakurai	63
Yuichi Nakamura	63

(10 rows returned; showing first 6)

Q5. Yearly trend of titles added

Date operations – EXTRACT(YEAR
FROM ...)

Problem: How has Netflix's catalog grown by year_added?

```
SELECT EXTRACT(YEAR FROM date_added)::INT AS yr, COUNT(*)  
FROM netflix GROUP BY yr ORDER BY yr;
```

year_added	titles_added
2010	2000
2011	2000
2012	2000
2013	2000
2014	2000
2015	2000

(16 rows returned; showing first 6)

Q6. Titles added in the last 5 years

Date operations – INTERVAL

Problem: How much of the catalog is recent, by content type?

```
SELECT type, COUNT(*) FROM netflix  
WHERE date_added >= (SELECT MAX(date_added) FROM netflix)  
      - INTERVAL '5 years' GROUP BY type;
```

type	recent_titles
Movie	5002
TV Show	5001

Q7. Top 10 highest-rated titles (vote_count ≥ 500)

Filtering + ORDER BY

Problem: What are the best-reviewed titles, once low-vote noise is filtered out?

```
SELECT title, type, vote_average, vote_count FROM netflix
WHERE vote_count >= 500 ORDER BY vote_average DESC LIMIT 10;
```

title	type	vote_average	vote_count
Arcane	TV Show	8.80	4932
Hazbin Hotel	TV Show	8.80	1229
Better Call Saul	TV Show	8.70	5475
The Owl House	TV Show	8.70	1643
Demon Slayer: Kimetsu no Yaiba	TV Show	8.70	6585
Rick and Morty	TV Show	8.70	9992

(10 rows returned; showing first 6)

Q8. Highest-rated title per content type

Window function – RANK() OVER
(PARTITION BY ...)

Problem: What is the #1 Movie and #1 TV Show by rating?

```
SELECT type, title, vote_average FROM (
  SELECT type, title, vote_average,
    RANK() OVER (PARTITION BY type ORDER BY vote_average DESC) rnk
  FROM netflix WHERE vote_count >= 500) x WHERE rnk = 1;
```

type	title	vote_average
Movie	Interstellar	8.50
Movie	Your Name.	8.50
Movie	Parasite	8.50
TV Show	Arcane	8.80
TV Show	Hazbin Hotel	8.80

Q9. Content originating from India

Pattern matching – ILIKE

Problem: How much India-linked content exists, and how is it rated?

```
SELECT type, COUNT(*), ROUND(AVG(vote_average),2)
FROM netflix WHERE country ILIKE '%India%' GROUP BY type;
```

type	titles	avg_rating
Movie	528	5.97
TV Show	384	4.38

Q10. Ratings & popularity: Movies vs TV Shows

Aggregation

Problem: Do Movies or TV Shows perform better on average?

```
SELECT type, ROUND(AVG(vote_average),2), ROUND(AVG(popularity),2),  
       ROUND(AVG(vote_count)) FROM netflix GROUP BY type;
```

type	avg_rating	avg_popularity	avg_vote_count
Movie	5.96	20.38	719
TV Show	5.42	64.85	107

Q11. Co-productions vs single-country titles

String manipulation –
ARRAY_LENGTH

Problem: How common are multi-country co-productions?

```
SELECT CASE WHEN ARRAY_LENGTH(String_to_array(country, ','),1) > 1  
       THEN 'Co-production' ELSE 'Single country' END, COUNT(*)  
FROM netflix GROUP BY 1;
```

production_type	title_count
Single country	24372
Co-production (2+ countries)	5365

Q12. Dynamic content categorization

Dynamic categorization – CASE +
ILIKE

Problem: Can we flag likely content themes directly from the description text?

```
SELECT CASE  
       WHEN description ILIKE '%kill%' OR description ILIKE '%war%' ... THEN 'Crime/Violence'  
       WHEN description ILIKE '%love%' THEN 'Romance' ... END, COUNT(*)  
FROM netflix GROUP BY 1 ORDER BY 2 DESC;
```

content_flag	title_count
Other	19238
Crime / Violence themed	5533
Romance themed	3388
Comedy themed	501

Q13. Movie profitability (budget vs revenue)

Aggregation + derived columns

Problem: Which movies were the most commercially successful?

```
SELECT title, release_year, budget, revenue, revenue-budget AS profit
FROM netflix WHERE type='Movie' AND budget>0 AND revenue>0
ORDER BY profit DESC LIMIT 10;
```

title	release_year	budget	revenue	profit	revenue_multiple
Avengers: Endgame	2019	356000000	2799439100	2443439100	7.86
Ne Zha 2	2025	80000000	1990000000	1910000000	24.88
Avatar: The Way of Water	2022	460000000	2320250281	1860250281	5.04
Star Wars: The Force Awakens	2015	245000000	2068223624	1823223624	8.44
Avengers: Infinity War	2018	300000000	2052415039	1752415039	6.84
Spider-Man: No Way Home	2021	200000000	1921847111	1721847111	9.61

(10 rows returned; showing first 6)

Q14. Top 10 most prolific directors

Aggregation + NULL handling

Problem: Which directors have the largest footprint on Netflix?

```
SELECT director, COUNT(*), ROUND(AVG(vote_average),2)
FROM netflix WHERE director IS NOT NULL AND director<>''
GROUP BY director ORDER BY 2 DESC LIMIT 10;
```

director	titles_directed	avg_rating
Tyler Perry	20	6.15
Gil Tejada Jr.	16	0.50
Don Michael Perez	16	1.66
Sean McNamara	15	6.28
Steven Soderbergh	15	5.82
Roel Reiné	15	5.53

(10 rows returned; showing first 6)

Q15. Most popular title per year (last 10 years)

Window function – RANK() OVER
(PARTITION BY release_year ...)

Problem: What was the single most-hyped release of each recent year?

```
SELECT release_year, title, type, popularity FROM (  
  SELECT *, RANK() OVER (PARTITION BY release_year  
    ORDER BY popularity DESC) rnk FROM netflix  
  WHERE release_year >= (SELECT MAX(release_year) FROM netflix)-9) x  
WHERE rnk=1 ORDER BY release_year DESC;
```

release_year	title	type	popularity
2025	The Gorge	Movie	3876.006
2024	Volta por Cima	TV Show	2885.720
2023	Die Landarztpraxis	TV Show	1898.550
2022	The Jennifer Hudson Show	TV Show	2066.124
2021	La Casa de los Famosos	TV Show	2126.920
2020	Lang Leve de Liefde	TV Show	1524.235

(10 rows returned; showing first 6)

Viva Prep — Questions Likely From These Design Choices

“Why did you use UNNEST instead of just counting rows?”

genres, country, and cast_members each hold multiple comma-separated values in one field. STRING_TO_ARRAY() converts that string into an array, and UNNEST() expands it into one row per value, so each individual genre/country/actor can be counted correctly instead of the raw combined string.

“Why generate a new primary key instead of using the dataset's own show_id?”

Because show_id collides across the two source files (397 shared IDs) and even within the TV shows file. A surrogate SERIAL key guarantees uniqueness; the original id is preserved as source_id.

“Why is there no 'content rating' (PG-13, TV-MA, etc.) in your analysis?”

This dataset doesn't include that classic categorical field — it instead provides a TMDB-style numeric quality score, popularity, and vote_count. We adapted the 'common ratings' problem into a rating-score and popularity analysis using those fields.

“Why filter on vote_count >= 500 in Q7/Q8?”

Without a minimum vote threshold, a title with one or two 10/10 votes would rank above titles with thousands of consistent high ratings — the filter avoids that small-sample bias.

“What does RANK() OVER (PARTITION BY ...) actually do here?”

It computes a rank within each group (e.g. within each type, or within each release_year) while keeping every row — unlike GROUP BY, which would collapse rows into one per group.

“How much of the 15 queries is actually 'done' versus just written?”

All 15 are written and have been executed successfully against the full real dataset — the results shown in this pack are live output, not samples. What's left for the final submission is the insights write-up and polishing the report document.